Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology

Department of

Electrical & Computer Engineering

Lab Report 5

Optimization of Neural Network Parameters Using Genetic Algorithms
(Neuro-Genetic Approach)

Course Code: ECE 4246

Course Title: Machine Learning Sessional

Submitted to:
Md. Faysal Ahamed
Assistant Professor
Dept of ECE, RUET

Submitted by:
Md. Tajim An Noor
Roll: 2010025
Semester: 4th Year Even

Submission Date: April 27, 2026

Contents

1 Theory and Introduction **1**
1.1 Chromosome Encoding 1
1.2 Fitness Function 1
1.3 Evolutionary Operators 2

2 Methodology **2**
2.1 Dataset 2
2.2 Algorithms 2
2.3 Code 2

3 Results and Discussion **6**
3.1 Quantitative Results 6
3.2 Visual Analysis 7

4 Conclusion **8**

References **8**

Optimization of Neural Network Parameters Using Genetic Algorithms (Neuro-Genetic Approach)

1 Theory and Introduction

Traditional Artificial Neural Networks (ANNs) rely on gradient-based optimization methods, such as Backpropagation and Gradient Descent, to update network weights and biases. While effective, gradient-based methods are prone to becoming trapped in local minima and require the activation functions to be strictly differentiable [1].

This experiment explores a derivative-free alternative: the **Neuro-Genetic Algorithm**. By framing the neural network parameters as a biological genome, Genetic Algorithms (GA) use evolutionary principles—Selection, Crossover, and Mutation—to iteratively optimize the network's weights.

1.1 Chromosome Encoding

To apply a genetic algorithm, the entire set of parameters for a given neural network must be encoded into a 1-dimensional array, known as a chromosome. For a network with weight matrices $W^{[1]}, W^{[2]}$ and bias vectors $b^{[1]}, b^{[2]}$, the chromosome C is a flattened concatenation:

$$C = \left[\text{flatten}(W^{[1]}), \text{flatten}(b^{[1]}), \text{flatten}(W^{[2]}), \text{flatten}(b^{[2]})\right] \tag{1}$$

1.2 Fitness Function

The fitness function evaluates how well a specific chromosome solves the classification problem. Instead of minimizing a cost function, the GA maximizes fitness. The fitness F is calculated using the inverse of the Binary Cross-Entropy (Log Loss) $J(C)$:

$$J(C) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log(\hat{y}^{(i)}) + (1-y^{(i)})\log(1-\hat{y}^{(i)})\right] \tag{2}$$

$$F(C) = \frac{1}{1+J(C)} \tag{3}$$

A lower loss yields a fitness closer to 1.0, while a higher loss yields a fitness closer to 0.

1.3 Evolutionary Operators

- **Tournament Selection:** A subset of individuals is randomly selected, and the one with the highest fitness is chosen to be a parent.
- **Uniform Crossover:** Two parent chromosomes exchange genes (weights) to produce offspring. A random binary mask dictates which parent contributes which gene.
- **Gaussian Mutation:** Random noise is injected into the genes of the offspring to maintain genetic diversity and explore new regions of the parameter space:

$$C'_{mutated} = C' + \mathcal{N}(0, \sigma^2) \quad \text{(for genes selected by mutation rate)} \tag{4}$$

2 Methodology

2.1 Dataset

The synthetic *Two Moons* dataset was utilized to evaluate the model's capacity to resolve non-linear classification problems. The dataset was standardized and split into 80% training and 20% testing sets.

2.2 Algorithms

Algorithm 1 Neuro-Genetic Optimization

Input: Training Data (X, Y), Population Size N, Generations G, Mutation Rate μ
Initialize Population: Generate N random chromosomes (weight arrays)
for $g = 1$ to G **do**
  for each chromosome C_i in Population **do**
    Decode C_i into $W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}$
    $\hat{Y} \leftarrow \text{ForwardPass}(X, C_i)$
    $Fitness(C_i) \leftarrow \frac{1}{1+\text{LogLoss}(Y,\hat{Y})}$
  end for
  Elitism: Keep the best performing chromosome
  $NewPopulation \leftarrow [BestChromosome]$
  while size of $NewPopulation < N$ **do**
    $P_1, P_2 \leftarrow \text{TournamentSelection}(Population, Fitness)$
    $Child \leftarrow \text{UniformCrossover}(P_1, P_2)$
    $MutatedChild \leftarrow \text{GaussianMutation}(Child, \mu)$
    Append $MutatedChild$ to $NewPopulation$
  end while
  $Population \leftarrow NewPopulation$
end for
return $BestChromosome$

2.3 Code

The algorithm was implemented natively in Python. The neural network architecture utilized 2 input neurons, 5 hidden neurons (with Tanh activation), and 1 output neuron (with Sigmoid activation), resulting in a chromosome length of 21 genes.

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons


# 1. EVALUATION & HELPER FUNCTIONS
def train_test_split_custom(X, y, test_size=0.2, seed=42):
    np.random.seed(seed)
    indices = np.random.permutation(len(X))
    split = int(len(X) * (1 - test_size))
    return (
        X[indices[:split]],
        X[indices[split:]],
        y[indices[:split]],
        y[indices[split:]],
    )


def standardize(X, mean_val=None, std_val=None):
    if mean_val is None or std_val is None:
        mean_val = np.mean(X, axis=0)
        std_val = np.std(X, axis=0)
        std_val[std_val == 0] = 1e-8
    return (X - mean_val) / std_val, mean_val, std_val


def get_confusion_matrix(y_true, y_pred):
    TP = np.sum((y_true == 1) & (y_pred == 1))
    TN = np.sum((y_true == 0) & (y_pred == 0))
    FP = np.sum((y_true == 0) & (y_pred == 1))
    FN = np.sum((y_true == 1) & (y_pred == 0))
    return np.array([[TN, FP], [FN, TP]])


def evaluate_metrics(y_true, y_pred):
    cm = get_confusion_matrix(y_true, y_pred)
    TN, FP, FN, TP = cm.ravel()

    accuracy = (TP + TN) / (TP + TN + FP + FN)
    precision = TP / (TP + FP) if (TP + FP) > 0 else 0
    recall = TP / (TP + FN) if (TP + FN) > 0 else 0
    f1 = (
        2 * (precision * recall) / (precision + recall)
        if (precision + recall) > 0
        else 0
    )

    return accuracy, precision, recall, f1, cm


# 2. PLOTTING FUNCTIONS
def plot_fitness_history(best_fitness, avg_fitness):
    plt.figure(figsize=(9, 5))
    plt.plot(best_fitness, color="green", linewidth=2, label="Best Fitness")
    plt.plot(
        avg_fitness,
        color="orange",
        linewidth=2,
        linestyle="--",
        label="Average Fitness",
    )
    plt.title("Genetic Algorithm Convergence: Fitness vs Generations")
    plt.xlabel("Generations")
    plt.ylabel("Fitness (1 / (1 + LogLoss))")
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()


def plot_confusion_matrix(
    cm, class_names=["Class 0", "Class 1"], title="Confusion Matrix"
):
    fig, ax = plt.subplots(figsize=(5, 4))
    im = ax.imshow(cm, interpolation="nearest", cmap="Greens")
    ax.figure.colorbar(im, ax=ax)

    ax.set(
        xticks=np.arange(cm.shape[1]),
        yticks=np.arange(cm.shape[0]),
        xticklabels=class_names,
        yticklabels=class_names,
        title=title,
        ylabel="Actual Label",
        xlabel="Predicted Label",
    )

    thresh = cm.max() / 2.0
    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            ax.text(
                j,
                i,
                format(cm[i, j], "d"),
                ha="center",
                va="center",
                color="white" if cm[i, j] > thresh else "black",
                fontsize=14,
            )
    fig.tight_layout()
    plt.show()


def plot_decision_boundary(X, y, model, title):
    plt.figure(figsize=(9, 6))

    x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
```

```python
    y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100), np.linspace(y_min, y_max, 100))

    mesh_points = np.c_[xx.ravel(), yy.ravel()]
    Z = model.predict(mesh_points)
    Z = Z.reshape(xx.shape)

    plt.contourf(xx, yy, Z, alpha=0.3, cmap="RdBu")
    plt.scatter(
        X[y == 0, 0],
        X[y == 0, 1],
        color="red",
        marker="o",
        edgecolors="k",
        label="Class 0",
    )
    plt.scatter(X[y == 1, 0], X[y == 1, 1], color="blue", marker="x", label="Class 1")

    plt.title(title)
    plt.xlabel("Feature 1 (Standardized)")
    plt.ylabel("Feature 2 (Standardized)")
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.show()


# 3. NEURAL NETWORK & GENETIC ALGORITHM
class NeuroGeneticAlgorithm:
    def __init__(
        self,
        input_size=2,
        hidden_size=5,
        output_size=1,
        pop_size=50,
        generations=200,
        mutation_rate=0.1,
    ):
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.output_size = output_size
        # Chromosome size = total number of weights and biases
        self.w1_size = input_size * hidden_size
        self.b1_size = hidden_size
        self.w2_size = hidden_size * output_size
        self.b2_size = output_size
        self.chromosome_length = (
            self.w1_size + self.b1_size + self.w2_size + self.b2_size
        )

        self.pop_size = pop_size
        self.generations = generations
        self.mutation_rate = mutation_rate

        self.population = []
        self.best_weights = None
        self.best_fitness_history = []
        self.avg_fitness_history = []

    def _sigmoid(self, z):
        return 1 / (1 + np.exp(-np.clip(z, -250, 250)))

    def _tanh(self, z):
        return np.tanh(z)

    def _forward_pass(self, X, chromosome):
        """Decodes chromosome into weights and runs forward propagation."""
        idx = 0
        W1 = chromosome[idx : idx + self.w1_size].reshape(
            self.input_size, self.hidden_size
        )
        idx += self.w1_size
        b1 = chromosome[idx : idx + self.b1_size].reshape(1, self.hidden_size)
        idx += self.b1_size
        W2 = chromosome[idx : idx + self.w2_size].reshape(
            self.hidden_size, self.output_size
        )
        idx += self.w2_size
        b2 = chromosome[idx : idx + self.b2_size].reshape(1, self.output_size)
        # Layer 1 (Hidden) - using Tanh for better non-linear mapping
        Z1 = np.dot(X, W1) + b1
        A1 = self._tanh(Z1)
        # Layer 2 (Output) - using Sigmoid for probability (0 to 1)
        Z2 = np.dot(A1, W2) + b2
        A2 = self._sigmoid(Z2)

        return A2

    def _calculate_fitness(self, X, y, chromosome):
        """Evaluates fitness based on Log Loss (Binary Cross Entropy)."""
        m = len(y)
        y = y.reshape(-1, 1)
        predictions = self._forward_pass(X, chromosome)
        epsilon = 1e-8
        # Log loss formula
        cost = (-1 / m) * np.sum(
            y * np.log(predictions + epsilon)
            + (1 - y) * np.log(1 - predictions + epsilon)
```

```python
        )
        # Fitness is inversely proportional to cost. Higher fitness = lower cost.
        fitness = 1.0 / (1.0 + cost)
        return fitness

    def _crossover(self, parent1, parent2):
        """Uniform Crossover: Randomly take genes from parent 1 or parent 2."""
        mask = np.random.rand(self.chromosome_length) > 0.5
        child = np.where(mask, parent1, parent2)
        return child

    def _mutate(self, chromosome):
        """Gaussian Mutation: Add random noise to genes based on mutation rate."""
        mutation_mask = np.random.rand(self.chromosome_length) < self.mutation_rate
        noise = np.random.randn(self.chromosome_length) * 0.5  # Scale of mutation
        chromosome[mutation_mask] += noise[mutation_mask]
        return chromosome

    def fit(self, X, y):
        print(f"Initializing Population ({self.pop_size} individuals)...")
        # Initialize population with random weights between -2 and 2
        self.population = [
            np.random.uniform(-2, 2, self.chromosome_length)
            for _ in range(self.pop_size)
        ]
        for generation in range(self.generations):
            # 1. Evaluate Fitness
            fitness_scores = [
                self._calculate_fitness(X, y, ind) for ind in self.population
            ]

            # Track best and average
            best_idx = np.argmax(fitness_scores)
            self.best_weights = self.population[best_idx]
            self.best_fitness_history.append(fitness_scores[best_idx])
            self.avg_fitness_history.append(np.mean(fitness_scores))
            # Print progress
            if (generation) % 20 == 0 or generation == self.generations - 1:
                print(
                    f"Generation {generation:3d} | Best Fitness: {fitness_scores[best_idx]:.4f} | Avg Fitness: {np.mean(fitness_scores):.4f}"
                )
            # 2. Selection (Tournament Selection)
            new_population = [
                self.best_weights
            ]  # Elitism: Always keep the absolute best
            while len(new_population) < self.pop_size:
                # Select 2 parents via tournament
                tournament_indices = np.random.choice(
                    self.pop_size, size=6, replace=False
                )
                parent1_idx = tournament_indices[:3][
                    np.argmax([fitness_scores[i] for i in tournament_indices[:3]])
                ]
                parent2_idx = tournament_indices[3:][
                    np.argmax([fitness_scores[i] for i in tournament_indices[3:]])
                ]
                # 3. Crossover
                child = self._crossover(
                    self.population[parent1_idx], self.population[parent2_idx]
                )
                # 4. Mutation
                child = self._mutate(child)
                new_population.append(child)

            self.population = new_population

    def predict(self, X, threshold=0.5):
        probs = self._forward_pass(X, self.best_weights)
        return (probs >= threshold).astype(int).flatten()


# 4. MAIN EXECUTION
def main():
    print("=" * 50)
```

```python
    print(" NEURO-GENETIC OPTIMIZATION ALGORITHM")
    print("=" * 50)
    # 1. Generate Non-Linear Data (Two Moons)
    print("Generating 'Two Moons' dataset...")
    X, y = make_moons(n_samples=600, noise=0.15, random_state=42)
    # 2. Split and Standardize
    X_train, X_test, y_train, y_test = train_test_split_custom(X, y, test_size=0.2)
    X_train_std, mean_val, std_val = standardize(X_train)
    X_test_std, _, _ = standardize(X_test, mean_val, std_val)
    # 3. Train Neuro-Genetic Model
    # Arch: 2 Inputs -> 5 Hidden -> 1 Output (17 weights/biases to evolve)
    print("\nStarting Genetic Evolution of Neural Network Parameters...")
    nga = NeuroGeneticAlgorithm(
        input_size=2,
        hidden_size=5,
        output_size=1,
        pop_size=60,
        generations=150,
        mutation_rate=0.1,
    )
    nga.fit(X_train_std, y_train)
    # 4. Predict & Evaluate on Test Set
    y_pred = nga.predict(X_test_std)
    acc, prec, rec, f1, cm = evaluate_metrics(y_test, y_pred)
    print("\n" + "=" * 40)
    print("EVALUATION REPORT (Test Set)")
    print("=" * 40)
    print(f"Accuracy:  {acc:.4f}")
    print(f"Precision: {prec:.4f}")
    print(f"Recall:    {rec:.4f}")
    print(f"F1 Score:  {f1:.4f}")
    print("\nConfusion Matrix:")
    print(cm)
    # 5. Visualizations
    plot_fitness_history(nga.best_fitness_history, nga.avg_fitness_history)
    plot_confusion_matrix(
        cm, class_names=["Class 0", "Class 1"], title="Neuro-Genetic Confusion Matrix"
    )
    plot_decision_boundary(
        X_test_std, y_test, nga, title="Evolved Decision Boundary (Test Data)"
    )


if __name__ == "__main__":
    main()
```

3 Results and Discussion

The Neuro-Genetic model was evolved over 150 generations with a population of 60 individuals. The performance progressively improved as favorable weights were selected and combined.

3.1 Quantitative Results

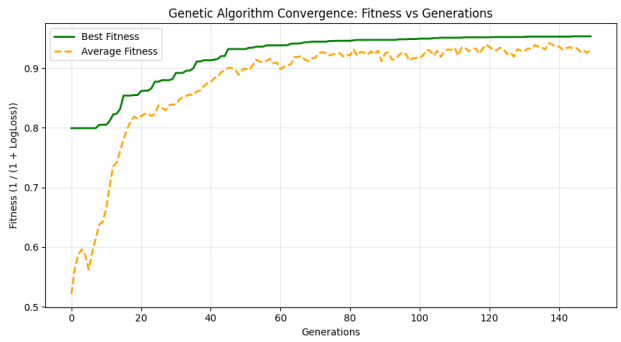
The console output logs demonstrate a steady increase in both the Best Fitness and Average Fitness of the population:

```
==================================================
 NEURO-GENETIC OPTIMIZATION ALGORITHM
==================================================
Generating 'Two Moons' dataset...
Starting Genetic Evolution of Neural Network Parameters...
Initializing Population (60 individuals)...
Generation   0 | Best Fitness: 0.7992 | Avg Fitness: 0.5208
Generation  20 | Best Fitness: 0.8614 | Avg Fitness: 0.8194
Generation  40 | Best Fitness: 0.9131 | Avg Fitness: 0.8764
Generation  60 | Best Fitness: 0.9380 | Avg Fitness: 0.8982
Generation  80 | Best Fitness: 0.9457 | Avg Fitness: 0.9211
Generation 100 | Best Fitness: 0.9493 | Avg Fitness: 0.9168
Generation 120 | Best Fitness: 0.9516 | Avg Fitness: 0.9352
Generation 140 | Best Fitness: 0.9527 | Avg Fitness: 0.9362
Generation 149 | Best Fitness: 0.9531 | Avg Fitness: 0.9316
========================================
EVALUATION REPORT (Test Set)
========================================
```

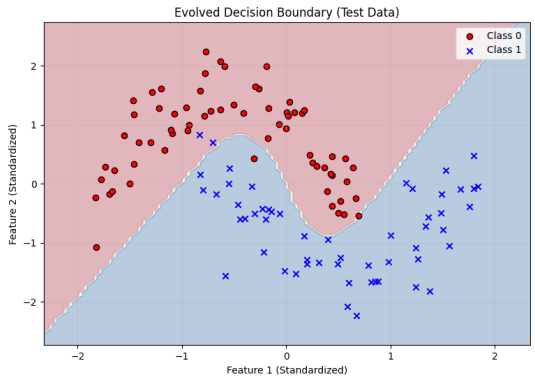
```
Accuracy:  0.9667
Precision: 0.9636
Recall:    0.9636
F1 Score:  0.9636
Confusion Matrix:
[[63  2]
 [ 2 53]]
```

The evaluation on the unseen test set yielded an excellent accuracy and F1 score of 96.67% and 0.9636, respectively. The confusion matrix (Figure 1c) confirms an evenly balanced error rate, with exactly 2 false positives and 2 false negatives.

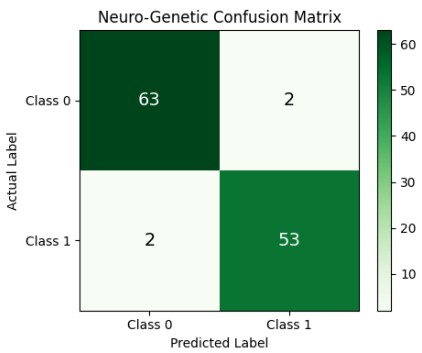
3.2 Visual Analysis



(a) Fitness Convergence vs. Generations



(b) Evolved Non-Linear Decision Boundary



(c) Neuro-Genetic Confusion Matrix

Figure 1: Neuro-Genetic Algorithm Performance Overview

- **Fitness Convergence (Figure 1a):** The convergence plot illustrates classic evolutionary behavior. The *Best Fitness* (solid green line) acts as an upper bound that incrementally steps up whenever a favorable mutation or crossover is discovered. The *Average Fitness* (dashed orange line) closely follows it, indicating that the overall genetic quality of the population rapidly stabilizes and improves over time.

- **Decision Boundary (Figure 1b):** The resulting neural network was able to map a highly non-linear boundary that accurately traces the "Two Moons" geometric distribution. This proves

that a derivative-free GA approach can successfully navigate a complex, 21-dimensional parameter space to find an optimal solution.

4 Conclusion

This experiment successfully demonstrated the viability of the Neuro-Genetic approach for training Artificial Neural Networks. By discarding traditional gradient descent in favor of evolutionary principles, the algorithm effectively optimized the network weights and achieved an accuracy of 96.67% on a complex non-linear dataset. While computationally expensive due to evaluating entire populations per generation, Genetic Algorithms provide a robust alternative for non-differentiable network architectures and complex loss landscapes where gradient methods might fail.

References

[1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning.* MIT press, 2016.